2. Solve 8-Puzzle using Breadth First Search (BFS) and Analyse

```
from collections import deque

def print_board(state):
    for i in range(0, 9, 3):
        print(state[i:i+3])
    print()


def get_neighbors(state):
    neighbors = []

    zero = state.index(0)

    moves = {
        0: [1, 3],
        1: [0, 2, 4],
        2: [1, 5],
        3: [0, 4, 6],
        4: [1, 3, 5, 7],
        5: [2, 4, 8],
        6: [3, 7],
        7: [4, 6, 8],
        8: [5, 7]
```

```
    }

    for pos in moves[zero]:
        new_state = state.copy()

        new_state[zero], new_state[pos] = \
            new_state[pos], new_state[zero]

        neighbors.append(new_state)

    return neighbors



def bfs(start, goal):

    queue = deque()

    queue.append((start, [start]))

    visited = set()
    visited.add(tuple(start))

    while queue:

        state, path = queue.popleft()
```

```
        if state == goal:
            return path

        for next_state in get_neighbors(state):

            state_tuple = tuple(next_state)

            if state_tuple not in visited:

                visited.add(state_tuple)

                queue.append(
                    (next_state, path + [next_state])
                )

    return None


start = [1, 2, 3,
         4, 0, 6,
         7, 5, 8]

goal = [1, 2, 3,
        4, 5, 6,
```

```
        7, 8, 0]

solution = bfs(start, goal)

if solution:

    print("Solution found using BFS")
    print("Number of moves:", len(solution) - 1)

    for state in solution:
        print_board(state)

else:
    print("No solution found")
```